

AXONA

Architecture

David A. Smith
Axona.net

An Axona Architecture Note v0.7.6 2026-06-03

A protocol is the contract between layers that don't trust each other.

What I cannot create, I do not understand.

RICHARD FEYNMAN · 1988

I The Stack

AXONA SHIPS AS THREE REPOSITORIES that compose into one running system. Each layer has a single responsibility,¹ a single canonical version identifier (`KERNEL_VERSION`), and a contract surface narrow enough to fit on a postcard.

@axona/protocol (kernel). The pure-JS library. Owns the DHT routing, the pub/sub overlay, the cryptography, and the transport contracts. No browser DOM, no Node-specific I/O, no network code outside abstract transport classes. Runs identically in browsers, in Node, in dht-sim’s simulated network.

axona-peer (browser application). The reference end-user app. Provides identity persistence, region selection, WebRTC mesh formation, a chat-style UI for pub/sub. Imports the kernel as a vendored copy under `vendor/axona-protocol/` so the version that ran when the page was built is the version that runs in the browser.

axona-bridge (deployed server). The signaling + introducer service at `bridge.axona.net`. Runs an embedded `AxonaPeer` of its own (so it’s a full DHT participant, not just a relay), gates connecting peers behind a version handshake, relays WebRTC signaling between browsers, and forwards Axona wire frames addressed to its own embedded peer.

The three components carry their own semver tracks because each ships to a different audience: kernel changes need API-level discipline; peer changes are user-visible UI; bridge changes are operational deploys. But they all share the same kernel version, visible at runtime through `/healthz` (bridge), the `welcome` frame (any peer connecting), and the version row in the peer UI.

II Identity and Addressing

EVERY AXONA PARTICIPANT has a 264-bit identifier. The top 8 bits are a Google S2 cell prefix² computed from the peer’s latitude and longitude; the bottom 256 bits are the SHA-256 of the peer’s Ed25519 public key. A peer in London has an ID starting with `0x44`; a peer

¹ The pattern is intentional. A peer’s business logic should never need to know the wire format; a bridge should never need to know what topic a subscriber cares about; the kernel should never need to know which application is using it. Every interface in this document is a place where one layer was given the right to be wrong about the layer below.

A fourth component, `dht-sim`, runs the kernel against a simulated network for protocol research and regression benchmarks. It’s not part of the production stack but shares the same `@axona/protocol` library; every change that ships to production also has to pass the simulator’s 25,000-node battery.

² S2 projects the sphere onto the six faces of a cube and partitions each face along a Hilbert curve. We bin to level 3 (8×8 per face, 384 cells globally), then truncate to 8 bits — equivalent to taking the top 8 bits of Google’s 64-bit level-3 cell ID. Result: 192 valid cells, with byte values 192–255 reserved for future system use. The cube projection keeps cell areas within $\sim 1.4\times$ of each other globally — no polar shrinkage — and any standard S2 library can recompute our prefixes by truncation.

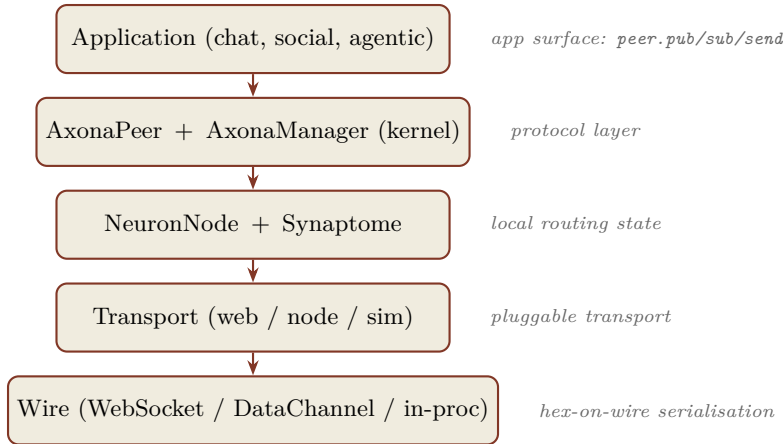


Figure 1: *

Five layers in one peer. Every arrow is a contract; every contract has a smoke-test suite.

in Virginia starts with 0x89. Two peers in the same region share a leading byte; two random peers share only the bits their keys happen to share.

The address space is treated as `BigInt` in every line of running code. The wire format is 66-character lowercase hex — the **ONLY** representation that ever appears in JSON. The conversion happens at exactly two places: when the transport serialises an outgoing frame, and when the dispatcher parses an incoming one.

Topic identifiers share the address space. A topic is named by the application (a string like `us-east/hello-world`) and anchored at a publisher: `topicId = sha256(publisher || topic-name)`, so its top byte is the publisher’s S2 cell. A regional synthetic publisher (top byte = S2 cell, bottom bits zero) anchors the topic in that region — which is what makes regional pub/sub route naturally: peers in the same region are XOR-closest to topics anchored there. A public topic (no publisher) is `00 || sha256(topic-name)`: the `00` prefix is simply the S2 cell at `00`, an ordinary cell with no special “global” meaning. Public topics are anchored there by convention, so every peer derives the same ID for a given name — and, like any topic, that ID is XOR-closest to the peers whose IDs fall in cell `00`.

The Activation Potential (AP) routing formula, the K-closest selector, and the synaptome’s stratum buckets all operate on this one space. There is no separate “topic ID space” or “peer ID space”; they share addresses because they share routing.

III The Kernel: @axona/protocol

THE KERNEL IS THE PROTOCOL. Everything else is its clients.
The directory structure mirrors the contract surface:

src/contracts/. Abstract base classes: **Transport**, **DHT**, **DHTNode**, **BootstrapService**, **PersistenceAdapter**. These are the seams where alternate implementations plug in. A custom transport (libp2p, QUIC, embedded) extends **Transport**; a database-backed persistence (IndexedDB, SQLite) extends **PersistenceAdapter**.

src/dht/. The active routing layer: **NeuronNode**, **Synapse**, **AxonaPeer**, **AxonaDomain**, **Subscription**. This is where the Axona routing logic lives, and where the developer-facing API surface (**peer.pub**, **peer.sub**, **peer.lookup**) is declared.

src/pubsub/. The pub/sub overlay: **AxonaManager**, **post.js** (topic ID derivation), **envelope.js** (signed publish envelopes), **ed25519.js** (Ed25519 binding for **crypto.subtle**). **AxonaManager** owns the K-closest tree, the replay caches, the dedup LRU, the metrics counters.

src/transport/. Concrete transports: **transport/web/** (browser: BridgeTransport over WebSocket + WebRTCTransport over DataChannels), **transport/node/** (server-side WebSocket), **transport/sim/** (in-process simulation for dht-sim and tests). All implement the abstract **Transport** contract identically.

src/utils/. Pure helpers: **hexid.js** (BigInt $\leftrightarrow$ hex conversions, XOR distance, leading-zero count), **s2.js** (S2 cell computation), JSON codec (**transport/wire.js**).

src/identity/. Ed25519 keypair derivation + persistence envelope. The single entry point **deriveIdentity** takes a **{lat, lng}** or full options object and returns **{id, pubkey, pubkeyHex, privateKey, region, ...}**.

The whole kernel has zero runtime dependencies beyond the host JavaScript environment (browser or Node 20+). No bundler, no transpilation, no build step. The same source files run in the browser via ES modules, in Node via direct **import**, and in the simulator via the in-process transport.

IV The Local Node

EACH PEER’S VIEW OF THE NETWORK lives in two data structures on the `NeuronNode`: the *synaptome* (its outgoing connections) and the *incoming synapse index* (peers known to point at it). Both are keyed by 264-bit `BigInt` `nodeId`.

A **Synapse** is a routing edge — it carries a peer ID, a measured latency, a learned weight, a stratum (the XOR-distance bucket), an inertia lock, and provenance (was this synapse added via handshake, hop-cache, triadic introduction, or sponsor bootstrap?). The synaptome is unbounded by design; the routing layer evicts synapses through the *Structural Survival Rule*, which guarantees navigability even under aggressive weight decay.

Routing decisions don’t pick the XOR-closest synapse blindly. Each candidate is scored by Activation Potential:

$$AP_c = \frac{\Delta \text{Distance}_c}{L_c} \times (1 + w \cdot W_c)$$

The first factor is progress velocity — how much XOR distance the hop closes per millisecond of measured latency. The second is a mild boost for synapses that have historically led to fast lookups. Weight only accrues on at-or-below-average-latency paths, so a high- W synapse genuinely represents a fast route, not just a frequently-used one. Two random hops with similar latency tie; the one that’s served us well before wins the tiebreak.

The rule, in one sentence: a synapse may only be pruned if at least one other synapse with the same stratum exists. The effect: every populated stratum keeps at least one live route, so the address space stays globally navigable even after heavy decay.

V The Protocol Layer

AXONAPEER IS THE ACTIVE OBJECT that turns a `NeuronNode` into a network participant. It owns:

- The handler installation. On `peer.start()` it registers the Axona routing handlers (`lookup_step`, `lookahead_probe`, `find_closest_set`, `route_msg`, `local_probe`, `triadic_introduce`, `hop_cache`, `lateral_spread`, `reinforce`) on the transport.
- The auto-admission loop. It asks `transport.bindPeers()` for currently-bound `nodeIds` and seeds them into the synaptome; it subscribes to `transport.onPeerBound` for future admissions.
- The pub/sub surface. `peer.pub`, `peer.sub`, `peer.pull`, `peer.metrics`: the developer-facing API. Each delegates to the `AxonaManager` after deriving the topic ID and (for `pub`) building a signed envelope.

- The direct-message surface. `peer.send`, `peer.notify`, `peer.onMessage`: per-peer request/response and fire-and-forget, outside the pub/sub overlay. For protocols that aren't topic-shaped.
- The lookup primitive. `peer.lookup(targetKey)` runs an iterative K-closest walk and returns `{found, path, hops, peerId, latency}`. This is what `AxonaManager` uses internally on every pub/sub to find the topic's axons.

`AxonaManager` is a separate object owned by the peer. Its job is the pub/sub overlay: when a subscriber sends `pubsub:subscribe-k` to the K-closest axons for a topic, the receiving `AxonaManager` stores the subscriber in its `role.children` for that topic. When a publisher fans `pubsub:publish-k` to its own K-closest, each receiving `AxonaManager` iterates its `role.children` and forwards `pubsub:deliver` to each subscriber. The publisher's signed envelope flows through unchanged; per-hop dedup prevents the fan-out tree from delivering duplicates.

The K-closest replication is what makes the system tolerant. A publisher sends to *five* root axons in parallel; a subscriber subscribes at *five* root axons in parallel. As long as one axon in each set overlaps, delivery succeeds. With $K = 5$ and synaptomes that converge under the bridge's peer-list dissemination, overlap is typically 4 or 5 of 5 — so a publish that loses two axons to transient WebRTC failures still delivers cleanly.

This section is the architectural placement of `AxonaPeer`; the field-by-field surface — every method, its parameters, and its return shape — is catalogued in the Application API section later in this document.

VI The Transport Layer

TRANSPORT IS WHERE ADDRESSES MEET SOCKETS. The kernel's `Transport` contract is a small surface: `start`, `stop`, `send` (request/response), `notify` (fire-and-forget), `isConnected`, `bindPeer`, `boundPeers`, `onPeerBound`, `onRequest`, `onNotification`, `onPeerDied`. Every concrete transport implements this surface and the upper layers don't care which one is wired in.

Three concrete implementations ship:

transport/web/. The browser stack. A `CompositeTransport` fans out to two sub-transports based on which one owns each `nodeId`: a `BridgeTransport` that carries Axona wire frames over the same `WebSocket` the bridge uses for signaling, and a `WebRTCTransport` over a `MeshManager` that owns the data-channel pool. The factory `webTransport({bridgeUrl, identity})` composes the two, drives the bridge's version gate, runs the hello/hello-ack admission, threads TURN credentials from the welcome frame into the mesh's ICE config, and resolves once everything is up.

The factory also owns the *connection lifecycle*, so an application doesn't hand-roll it: a 1 Hz ping/pong heartbeat with stale detection, auto-reconnect with exponential backoff on an unexpected socket close (re-running the version gate and hello/hello-ack on each re-open), and a small observability surface — a `bridgeState` machine (`connecting` → `open` → `stale` → `disconnected`, plus `upgrade-required` on a 4426 close), the captured `welcome` (`bridgeInfo`), and the live ping RTT (`bridgeRtt`). A 4426 version-gate rejection suppresses reconnect; `reconnectNow()` forces an immediate re-attempt (used on tab-resume and network-online).

The WebRTC mesh under `webTransport` self-heals on the same principle. `MeshManager` runs a 1 Hz ping/pong on every data channel and (kernel v2.1.1) evicts a peer the moment its channel goes dead: no pong for the heartbeat-timeout window, or a few consecutive throwing `send()`s — the signature of a channel a browser keeps reporting `open` after a laptop sleep, where the socket is gone but `readyState` lies. Eviction fires `onPeerLost` → `onPeerDied`, so the protocol layer drops the synapse and routes around the dead peer while the bridge's next peer-list rebuilds a fresh channel. This is the Transport contract's heartbeat-timeout obligation, and it means recovery from a connection freeze needs *zero* application code. An application *may* add an accelerator: on a resume event (`visibilitychange`, `online`, `bfcache` `pageshow`) call `transport.mesh.reset()` + `reconnectNow()` to clear stale peers at once rather than waiting out the per-peer timeouts — both the demo and the axona.net peer do this. A macOS screensaver notably fires none of those resume events (the tab never goes hidden), which is why the transport-level eviction is the load-bearing mechanism and the app-level reset is only a speed-up.

transport/node/. A Node-side WebSocket transport. Used by the bridge's embedded peer to talk to connected browsers without simulating WebRTC — the bridge *is* the signaling server, so it doesn't need WebRTC to reach any browser; the WebSocket is direct.

transport/sim/. The simulator transport. Used by dht-sim's `TransportAxonaEngine` and by the kernel's smoke tests. Identical Transport contract, but messages are scheduled in the event loop with simulated latency derived from a real internet topology dataset.

The wire format is JSON, with a single quirk: `BigInt` is serialised as `"<digits>n"` (decimal digits with an `n` suffix) and revived back to `BigInt` on decode. `NodeId` fields are always hex strings on the wire —

The same kernel test suite runs against all three transports. Cross-transport bugs — which is to say, the kind of bug where a fix lands in one transport's call path but not the others — can't ship.

they never appear as bigint-suffixed literals because the kernel converts them at frame construction.

VII The Bridge

THE BRIDGE IS AN INTRODUCER, not a server. Its job is to let browsers find each other; once they've found each other, WebRTC data channels carry every byte of application traffic peer-to-peer. The bridge stays in the picture only for two reasons: (1) it's the rendezvous for the version handshake before peers can talk at all; (2) it's a guaranteed-reachable mesh peer, which makes it a useful K-closest axon in small networks where two browsers might otherwise fail to find common axons.

The bridge handles three kinds of frames over the WebSocket:

Signaling. `welcome`, `peer-list`, `peer-joined`, `peer-left`, `signal`, `ping/pong`. Routes WebRTC SDP offers/answers and ICE candidates between browsers, broadcasts join/leave events, supplies short-lived TURN credentials in the welcome frame.

Version handshake. `client-hello` / `server-hello`. Enforces a minimum peer version. Rejects sub-minimum peers with WebSocket close code 4426 and a human-readable reason; the peer's UI shows a "please reload" banner.

Axona wire. `{type: 'axona', payload: ...}` frames carrying `req/res/ntf` messages between any connected browser and the bridge's embedded peer. These are the same frames any two browsers exchange over WebRTC — the bridge embeds a full `AxonaPeer` of its own (`BridgeAxonaNode`) and participates in routing just like any other node, which is why it can be in K-closest sets and why it can hold subscriber children for pub/sub topics.

The bridge's embedded peer has a fixed, disk-persisted identity (895bf0... in production, anchored in us-east — the 0x89 S2 cell, which is why it sits XOR-close to `us-east/*` topics and is so often a K-closest root). Its `health` surface exposes the kernel version it's running and the synaptome it has built up, available at `/healthz` JSON. Operators check that endpoint to confirm a deploy.

How a New Node Joins

JOINING IS A MUTUAL, AUTHENTICATED BIND. A fresh node has no peers; it reaches the mesh through a bridge and then fans out

The kernel builds a peer's `AxonaManager` lazily, on its first `pub/sub`. The bridge never publishes or subscribes — it only relays — so it must build that engine *eagerly* at startup, otherwise the `pubsub:subscribe-k/publish-k` handlers are never installed and every `pub/sub` frame addressed to the bridge is silently dropped. Because the bridge sits in nearly every peer's K-closest set, that failure mode is a near-invisible delivery hole; the eager build is what keeps it an honest relay axon.

to direct WebRTC links. Crucially, synaptome membership is established the *same* way on every link — bridge or mesh — so “joining” is just the first instance of the generic admit path, not a special case.

1. **Connect + version gate.** The node opens a WebSocket to a bridge and exchanges `client-hello/server-hello`. A sub-minimum peer is closed with 4426 before it can do anything else.
2. **Welcome.** The bridge replies with `welcome`, carrying the connection id and a fresh `serverNonce`; both sides now hold the per-connection channel-binding value (CBV) the next step signs over.
3. **Authenticated handshake (axona/4).** Each side sends a `hello` (the responder a `hello-ack`) carrying its Ed25519 public key and a signature over the link’s CBV, and verifies the other: the presented pubkey must hash to the 256-bit suffix of the `nodeId` it claims, and the signature must check out. A peer that cannot prove its id — or one being MITM’d, since the CBV binds the proof to *this* channel — fails here and is closed.
4. **Mutual bind + admit.** On success both ends record the binding and admit the other into their synaptome *as an ordinary Synapse*: the joining node calls `bindPeer(bridgeId, 'bridge')`, whose `onPeerBound` hook seeds the bridge into its synaptome (`addedBy: bootstrap`); the bridge’s embedded peer runs `_completeHandshake`, binds the new node to that connection, and admits it via `_addByVitality` (`addedBy: handshake`). No bridge-specific code runs in either step — the bridge is simply the first guaranteed-reachable peer the node binds.
5. **Fan out to the mesh.** The bridge broadcasts `peer-joined` and hands the new node a `peer-list`. The node negotiates WebRTC data channels to those peers (SDP relayed via the bridge), and each channel runs the *same* axona/4 hello — now additionally bound to the peer’s DTLS-certificate fingerprint — firing the same `onPeerBound` admit. The synaptome fills with direct mesh peers; the bridge stops being the only neighbour.
6. **Converge.** Optionally `peer.join(sponsor)` seeds a specific first neighbour; thereafter vitality (`weight × recency`) and the 10 s `refreshTick` grow and prune the synaptome and re-anchor subscriptions onto the converged K-closest set. The node is now a full participant.

The admit is symmetric and identity-bound in both directions — each side admits the other’s *authenticated* `nodeId`, so a peer can only ever be admitted under an id it proved it owns. Because every node runs step 4 against the bridge first, the bridge lands in every synaptome: its centrality is *emergent* from this bootstrap, not granted by any special rule (see § Future Directions, “demote the bridge to an ordinary peer”).

VIII The Wire Protocol

EVERY FRAME ON THE WIRE is JSON. Every `nodeId` or `topic ID` field is a 66-character lowercase hex string. There's no binary framing, no protobuf, no length-prefixed records — the discipline of having one wire format that any of the three transports can carry is worth the few percent of size you give up.

Three kinds of message-typed envelopes flow over the wire:

Request (*k*: `'req'`). A correlated request expecting a response.

Carries an integer `id` for correlation, a `type` string naming the handler (`lookup_step`, `route_msg`, `find_closest_set`, ...), and an opaque `body`.

Response (*k*: `'res'`). The reply to a `req`. Echoes the `id` and adds an `ok` boolean and a `body`.

Notification (*k*: `'ntf'`). Fire-and-forget. No `id`, no response. Used for `hello/hello-ack` admission, `pubsub:subscribe-k`, `pubsub:publish-k`, `pubsub:deliver`, `triadic_introduce`, `hop_cache`, `lateral_spread`, `reinforce`.

The handler types are versioned implicitly through the kernel — the bridge's `minPeerVersion` gate excludes peers running an older protocol that wouldn't understand newer messages. Major protocol changes bump `KERNEL_VERSION`'s minor; minor changes are additive (a peer that doesn't know a handler type silently drops the message).

A pub/sub message envelope is signed:

```
{
  msgId:      "<sha256 of canonical(envelope) hex>",
  seq:        1716638400123,    // per-publisher monotonic (v2.9.0)
  ts:         1716638400000,
  topic:      "us-east/hello-world",
  message:    "<application payload>",
  signerPubkey: "<Ed25519 pubkey hex>",
  signature:  "ed25519:<128 hex chars>"
}
```

Receivers verify the signature against the signer's pubkey using `crypto.subtle.verify`; the kernel surfaces verification result on the delivered envelope so applications can decide whether to trust unsigned or unverifiable publishes.

As of kernel v2.9.0 (envelope format **v2**, finding C-2) the signature covers a domain-separated core (`axona:pubsub-envelope:v2`) and

includes a per-publisher monotonic **seq**. Two properties follow. *Freshness*: a topic’s root axons reject a *live* publish whose signed **ts** is outside a ± 5 -minute window, so a captured envelope can’t be re-injected as new once it ages out (the legitimate replay-to-late-subscribers path is exempt — it serves cached history). *Order*: **seq** gives each publisher’s stream a total order, which is what makes “latest” and bounded-queue eviction (below) deterministic across replicas.

IX Routing

AXONA’S ROUTING IS LEARNING-ADAPTIVE. It replaces Kademlia’s k -bucket array with a dynamic synaptome that adjusts edge weights based on observed latency. The basic shape — iterative lookup, $\log_2(N)$ hop count — is unchanged from Kademlia; what changes is which peer you ask at each hop.

The five wire operations that drive Axona routing are:

lookup_step (*req*). “Here’s a target. What’s your best next hop?”

The receiver evaluates AP across its synaptome’s progress candidates, returns the winning synapse + its own routing state.

lookahead_probe (*req*). “If I forward to you for target T , who would you forward to?” A one-deep peek that lets the caller compare expected hop costs across candidates before committing.

find_closest_set (*req*). “Give me your K peers closest to this target.” The K-closest harvester used by AxonaManager’s pub/sub axon discovery.

route_msg (*req*). Generic routed-message forwarder. Used by AxonaManager for **__tunneled_direct__** delivery when the target isn’t directly bound.

reinforce (*ntf*). “This lookup succeeded; bump the weight on the synapse I used.” Fire-and-forget LTP signal back along the path. Only flows when the lookup hit at-or-below average latency for the path’s region. On identity-binding transports the handler is a no-op unless the *target* synapse’s peer is currently bound (the B-3 gate: an unauthenticated **reinforce** must not be able to refresh the eviction-inertia of a stale or unverified entry and pin it in the table). Weight is independently clamped to ≤ 1.0 , so inertia is the only lever the gate needs to close.

Two structural-plasticity messages let the network rewire itself without explicit instruction: **triadic_introduce** (“introduce me to your friend at X ”) and **hop_cache** (“I’ve forwarded to X several times;

Residual, by design: the gate keys on the *target* synapse being bound, not on the *sender*’s identity — so any bound peer can refresh the inertia of any other bound peer’s synapse. The blast radius is bounded to “keep an already-verified, live peer pinned slightly longer,” which is benign; binding the signal to the sender would add cost for no attacker-relevant gain. Revisit only if reinforce ever carries weight a sender could bias.

install a direct synapse to X). Both are fire-and-forget notifications that obey the synaptome’s admission rule (`_addByVitality`).

X Pub/Sub: K-Closest

THE PUB/SUB OVERLAY IS ITS OWN STRUCTURE. It sits on top of Axona routing but doesn’t reuse the synaptome for state — each peer maintains a separate set of *axon roles*, one per topic it’s chosen as an axon for. A role holds a topic ID, a set of children (subscribers), a replay cache, and a K -replicated peer set (the other root axons for the same topic).

Subscriber-side flow:

1. Compute `topicId = sha256(publisher || topic-name)`.
2. Find the K peers closest to `topicId` in the local synaptome. (No network walk — the local reachable peer set is what matters; remote-walk K -closest returns ghost IDs that route fan-out into dead ends.)
3. Send `pubsub:subscribe-k` to each, carrying the subscriber’s `nodeId` and an optional `lastSeenTs` for replay.
4. Each receiver runs `_addOrRecruitChild`, stores the subscriber under `role.children`, and optionally serves a `pubsub:replay-batch` of cached messages newer than `lastSeenTs`.

Publisher-side flow:

1. Compute the same `topicId`.
2. Build a signed envelope.
3. Find the same K peers closest to `topicId` locally.
4. Send `pubsub:publish-k` to each.
5. Each receiver runs `_onPublishDirect`: dedup against `_seenPublishes`, add to replay cache, fan `pubsub:deliver` to every child in `role.children`.
6. Each child receives `pubsub:deliver`, dedups, fires the local `_deliveryCallback` which routes to the application’s subscription handler.

When a publisher’s K -closest peers also happen to be in the subscribers’ K -closest sets, the tree converges and delivery is single-hop from publisher to axon to subscriber. When they don’t, the system relies on the redundancy: a missing axon’s subtree is covered by the four other axons. The Activation Potential weighting makes high-quality axons dominate the K -closest set over time.

The performance claim of Axona over Kademlia is roughly 40% fewer wall-clock milliseconds per lookup at 25,000 nodes, mostly from latency-weighted hop selection. The figure is documented in the companion paper *Axona: A Learning-Adaptive Distributed Hash Table and Axonal Publish/Subscribe* (v0.4.01, 2026-05-23).

Per-hop `_alreadySeenPublish` dedup is keyed on `publishId` (a string like `nodeId:counter`), separate from the content-hash `msgId` the application sees. Multiple arrivals of the same `publishId` at one peer collapse to a single `_deliveryCallback` firing.

K-CLOSEST IS COMPUTED AGAINST THE LOCAL SYNAPTOME, so in a browser mesh that fills in gradually the answer is a moving target: a peer that subscribed while it only knew the bridge picked a narrow axon set, and a publisher that joined later computes a wider one — they can fail to overlap, and the publisher misses that subscriber. The result is cached per topic under an epoch counter to avoid recomputing on every `pub/sub`; the fix is to *invalidate* that cache as the mesh changes. `AxonaPeer` bumps the epoch on every `onPeerBound`, and `AxonaManager.refreshTick` (a 10s timer the application arms after subscribing) flushes it and re-issues every live subscription against the now-converged set. So a subscription anchored early re-anchors as the mesh widens, rather than staying pinned to a stale, narrow axon set.

Topic Metrics: Scatter-Down, Gather-Direct

`PEER.METRICS(TOPIC)` reports a topic's live state — `publishes`, `current_count` (events retained in the tree right now), `subscribers`, `deliveries`, `pulls`, `reshares` — by querying the same axon tree that carries delivery. It is a *scatter-down, gather-direct* traversal, not a recursive sum back up the tree.

1. **Route to the tree.** `requestMetrics` issues a routed `pubsub:metricsReq` toward the `topicId`; a node that doesn't host the topic returns `'forward'`, so the request DHT-routes until it reaches a root axon.
2. **Scatter down.** On receipt a relay replies with its *own* local view, then forwards the request as `pubsub:metricsBroadcast` to each of its children, which recurse — the probe walks the tree along the same parent→child edges that fan messages out. A bounded seen-request set (keyed on `requestId`) makes each relay process a given probe once, so redundant edges neither loop nor double-count.
3. **Gather direct.** Every responding relay sends its partial result *straight back to the original requester* (`pubsub:metricsResp`), not up through its parent. Scattering down but gathering directly keeps the root from becoming an aggregation hotspot.
4. **Bounded collection.** The requester accumulates responses for a fixed window (`timeoutMs`, default 500 ms) — there is no global barrier — then folds them. `relayCount` (distinct responders) is the honest coverage signal; the result is a best-effort snapshot, not a ledger.

WHY THE FOLD DIFFERS PER FIELD. Each relay holds a local slice: its replay cache, its `children` map, and per-post counters. The

tree replicates the message queue across the K root axons but partitions subscribers across the tree, so a uniform sum would be wrong. The client combines each quantity according to its nature.

Field	Fold	Why
<code>publishes</code>	max	distinct posts are replicated across roots; summing would multiply by K
<code>current_count</code>	max	replicas hold the same retained set; max also tolerates a lagging replica
<code>deliveries,</code> <code>pulls,</code> <code>reshares</code>	sum	partitioned work counters — each relay tallies only the actions it performed
<code>subscribers</code>	max	membership is partitioned; exact for an un-split (single-root) topic, a lower bound once the tree splits
<code>relayCount</code>	count	distinct responders — a coverage indicator

Replicated set-cardinality folds with max; partitioned per-action counters fold with sum; partitioned membership folds with max (approximate).

A self-authenticating ownership gate means a relay answers only when the topic's publisher matches the requester, so today the owner alone gets non-empty results. `current_count` and live `subscribers` were added in kernel v2.11.0.

XI The Application API

EVERY AXONA APPLICATION TALKS TO THE PROTOCOL THROUGH ONE IMPORT. The barrel export at `@axona/protocol` is the entire contract surface — a small set of factory functions, one peer class, and a handful of helpers. This section is the field-by-field reference.

Identity

`deriveIdentity({ lat, lng })` *generate a fresh peer identity*

`lat` number. Latitude in decimal degrees.

`lng` number. Longitude in decimal degrees.

Returns a Promise resolving to `{ id, pubkey, pubkeyHex, privkey, privateKey, region }`. `id` is the 66-char hex `nodeId` (8-bit S2 prefix from `(lat,lng)` + 256-bit SHA-256 of the public key). `privateKey` is the in-memory `CryptoKey` ready for signing; `privkey` is the base64-encoded export for persistence. The whole envelope is what `AxonaPeer` expects as `identity`.

`dumpIdentity(identity)` *serialize for storage*

Returns a Promise resolving to a plain JSON object you can write to disk or IndexedDB. The reciprocal `loadIdentity(envelope)` reconstructs the `CryptoKey`-bearing form on the next launch.

Transports

webTransport(`{ ... }`) *browser transport: WebSocket to the bridge + WebRTC mesh*

bridgeUrl string. e.g. `wss://bridge.axona.net`.

identity identity envelope from `deriveIdentity`.

log (*optional*, *default* `() => {}`) function (`event`, `data`) that receives transport-internal diagnostic events.

WebSocketImpl (*optional*) constructor for the `WebSocket` class. Defaults to `globalThis.WebSocket`.

autoHandshake (*optional*, *default* `true`) when `true`, `transport.start()` drives the full bridge admission sequence end-to-end — the WebSocket version gate, the application-level `hello/hello-ack` exchange, and binding the bridge as a peer. Set to `false` only if you're driving the handshake yourself.

peerVersion (*optional*, *default* `KERNEL_VERSION`) semver string sent in `client-hello`.

handshakeTimeoutMs (*optional*, *default* `15000`) how long to wait for the bridge's `hello` before rejecting the *first* `start()`.

reconnect (*optional*, *default* `true`) auto-reconnect with exponential backoff on an unexpected socket close (bounds tunable via `reconnectInitialMs` / `reconnectMaxMs`). A 4426 version-gate rejection is the one close that does *not* reconnect.

Returns a `CompositeTransport`. After `await transport.start(identity.id)` the bridge is reachable as a peer (`transport.bridgeNodeId`) and the WebRTC mesh is ready to accept channels. Beyond the `Transport` contract the returned object carries the connection-observability surface the factory now owns:

onBridgeState(*cb*) subscribe to state transitions `connecting | open | stale | disconnected | upgrade-required`; `bridgeState` is the current value, `upgradeReason` the 4426 detail.

onWelcome(*cb*) the bridge's `welcome` (`{connId, version, kernelVersion, turn}`), also cached on `bridgeInfo` and replayed to late subscribers.

bridgeRtt / **bridgeRttAvg** last and windowed ping→pong round-trip in ms.

onPingTraffic(*cb*) per-frame (`nodeId`, `'sent' | 'recv'`) pulses for live link indicators.

reconnectNow() force an immediate reconnect with the backoff reset (`tab-resume` / `network-online`).

`nodeTransport.server({ ... })` *server-side WebSocket transport (Node 20+)*

`nodeTransport.client({ ... })` *Node WebSocket client*

Used by `axona-bridge`'s embedded peer and by Node applications.

Same `Transport` contract as the browser transport; no WebRTC.

`simTransport({ network, identity })` *in-process simulation transport*

network `SimNetwork` shared across peers.

identity identity envelope.

For tests and `dht-sim`. Messages are scheduled in the event loop with synthetic latency from a real internet topology dataset.

AxonaPeer

`new AxonaPeer({ ... })` *the per-node DHT instance*

node `NeuronNode` instance for this peer (carries id, lat, lng, the synaptome).

domain `AxonaDomain` the peer joins. Construct one per network with `new AxonaDomain({ k: 20 })`, where `k` (*default 20*) is the *K*-replication factor.

transport the transport instance from `webTransport`, `nodeTransport`, or `simTransport`.

identity (*optional*) identity envelope. Required for signed publishes (the default). Apps that only call `peer.pub(t, m, { sign: false })` can omit it.

axonaManager (*optional*) explicit `AxonaManager`. When omitted, one is built automatically on `peer.start()`.

persist (*optional*) `PersistenceAdapter` for synaptome / subscription / replay-cache checkpoints. Default: no persistence.

Lifecycle

`await peer.start()` *wire handlers, install delivery hook*

`await peer.stop()` *detach handlers and event listeners*

`await peer.join(sponsor?)` *production bootstrap path*

sponsor (*optional*) 66-char hex node ID. If given, opens a transport channel to that peer and seeds the synaptome from it. If omitted, returns immediately — the peer is “joined” but isolated; inbound connections from other peers populate the synaptome.

`await peer.leave(opts?)` *leave the network gracefully*

`opts.drain` (default `true`) wait for in-flight publishes to settle.

`opts.notify` (default `true`) send a `peer-leaving` notification to every peer in the synaptome.

`opts.timeoutMs` (default 5000) maximum drain window in ms.

Pub / Sub

`await peer.pub(topic, message, opts?)` *publish to a topic*

`topic` string. Application-level topic name (any non-empty string).

`message` JSON-serializable payload.

`opts.sign` (default `true`) sign the envelope with the identity's Ed25519 key.

`opts.publisher` (default: *this peer's nodeId*) controls where the topic is anchored in the DHT. `undefined` means this peer publishes its own topic; `null` means the public well-known mode (`topicId=SHA256(topic)`, anyone can publish); a 66-char hex string lets a peer relay under that publisher's anchor (rare).

Returns a Promise resolving to the `msgId` — the 64-char hex content hash of the envelope. Throws `PublishError` on invalid inputs or signing failure.

`await peer.sub(topic, handler, opts?)` *subscribe to a topic*

`topic` string.

`handler` function (`envelope`) => void invoked for each delivery.

The envelope is { `msgId`, `seq`, `ts`, `topic`, `message`, `publisher`, `signerPubkey` }. A retraction (see `peer.kill`) arrives on the same handler as a marker, { `deleted: true`, `msgId`, `topic` } — branch on `env.deleted` to drop your local copy.

`opts.publisher` (default: *this peer's nodeId*) same semantics as `pub` — must match the publisher's `publisher` value for the subscription to land at the right topic ID.

`opts.since` (default: *live tail*) replay control. Pass `'latest'` for the most-recent cached message plus future deliveries, `'all'` for the full replay cache plus future, or a Unix-ms `number` for everything newer than that timestamp.

Returns a Promise resolving to a `Subscription` handle. Call `sub.stop()` to cancel that handle, or `peer.unsub(topic, opts)` (kernel v2.10.0) to leave the topic by name — stops every local subscription for it

and, when the last goes, sends the network unsubscribe (self-only by construction).

`await peer.pull(msgId, opts)` *fetch a message from the relay's replay cache — by ID, or the latest*

msgId 64-char hex content hash, or `null`/omitted to fetch the topic's most-recent message (highest `seq`; kernel v2.10.0).

opts.topic string. The topic name (required, used to locate the relay).

opts.publisher 66-char hex publisher ID, or `null` for public topics.

opts.timeoutMs (default 1000).

Returns the envelope or `null` if not found (which includes a message that has *expired* past its hold time). A successful pull slides the message's hold forward, bounded by its 48 h ceiling.

`await peer.metrics(topic, opts)` *aggregate counters across the relay tree*

topic string.

opts.publisher 66-char hex publisher ID, or `null`.

opts.timeoutMs (default 500).

Returns { publishes, subscribers, deliveries, pulls, reshares, relayCount }.

Message lifecycle (kernel v2.10.0)

Three lifecycle controls, each **self-authenticating** — the authority to act is proven by the same keypair that proved authorship/ownership, with no registry or central gatekeeper.

`await peer.kill(topic, msgId, opts?)` *retract a message you published (creator-only)*

A topic's root axons accept the kill only if it is signed by the *same key that signed the original message*; they drop it from their replay caches, record a short-lived tombstone (so a lagging replica can't resurrect it), and forward a delete marker to subscribers. It is best-effort redaction, *not* a cryptographic un-send — a subscriber that already received the plaintext can keep it — and an anonymous (`sign: false`) message has no provable creator and so cannot be killed. Throws `KillError`.

`await peer.unpub(topic, opts?)` *remove a topic's whole queue (owner-only)*

The *owner* is the identity whose `nodeId` seeds the topic ID; roots verify this self-authenticatingly (the signer's pubkey must bind to the owner `nodeId` *and* that `nodeId` must derive the topic ID). `opts.destroy`

(default **false**) escalates from “clear the messages” to total removal (messages *and* config/ownership state). Public (ownerless) topics cannot be unpublished. Throws **UnpubError**.

Topic limits

Each topic’s replay queue is bounded (kernel v2.10.0): **max messages** (default 100, ceiling 256) with deterministic eviction of the lowest-ordered message by signed (**seq**, **ts**, **msgId**) — a max of 1 is a retained / latest-value slot; a **hold time** (default 24 h, hard ceiling 48 h) after which a message expires and is swept; and, on *open* (anyone-may-publish) topics, a **per-publisher quota** (one-quarter of the queue) so a single publisher can’t flood out the rest. Owner-gated topics are governed by their publish rules instead.

Direct messaging

`await peer.send(targetId, message)` *RPC-style; awaits the remote handler’s return value*

`peer.notify(targetId, message)` *fire-and-forget*

`peer.onMessage(handler)` *register a handler for incoming direct messages*

targetId 66-char hex node ID. The peer must already be in the synaptome.

message JSON-serializable payload.

handler function (`fromId, message`) => `result`. The return value is sent back to the caller on **send**; ignored on **notify**. Async handlers are awaited.

Mesh introspection

`peer.peers()` returns an array of 66-char hex node IDs currently in the synaptome.

`peer.onPeerJoin(handler)` registers (`nodeId`) => `void` for new arrivals.

`peer.onPeerLeave(handler)` registers (`nodeId`) => `void` for departures.

`peer.health()` returns { `started`, `transportConnected`, `synapseCount`, `subscriptionCount`, `kernelVersion` }.

`peer.onLog(level, handler)` `level` is 'info', 'warn', or 'error'.

`peer.onError(handler)` registers (`err`) => `void` for unhandled errors.

Lookup

`await peer.lookup(targetKey)` *iterative K-closest walk*

targetKey 66-char hex 264-bit key. Usually a topic ID or a node ID.

Returns { **found**, **path**, **hops**, **peerId**, **latency** }. **found** is the closest reachable peer; **path** is the list of nodes traversed; **hops** is the count; **latency** is wall-clock ms.

Snapshots

`await peer.snapshot()` returns a JSON object capturing identity, synaptome, axon roles, subscriptions, replay caches. Symmetric with `AxonaPeer.fromSnapshot(snap, deps)`, which reconstructs a peer from one of these dumps without touching the network.

Helpers

`await deriveTopicId(publisher, topic)` returns the 66-char hex topic ID. **publisher** may be `null` (public mode) or a 66-char hex node ID. Useful for displaying the publish location in a UI.

`geoCellId(lat, lng, bits)` returns the integer S2 cell at the given precision. **bits** = 8 matches the top 8 bits of every Axona `nodeId`.

XII Identity and Security

IDENTITY IS AN ED25519 KEYPAIR. The kernel uses `crypto.subtle.generateKey` to produce a curve-Ed25519 pair; the public key is hashed with SHA-256 to produce the bottom 256 bits of the `nodeId`; the top 8 bits are the S2 cell of the user's (`lat`, `lng`). The private key never leaves the host environment, but the public key is broadcast on every connection (in `hello/hello-ack`) so peers can verify each other.

Every signed publish carries the publisher's public key (`signerPubkey`) inline. The kernel verifies the `ed25519:<hex>` signature against `signerPubkey` using `crypto.subtle.verify`. Verification is per-envelope, not per-peer — a peer that's untrusted can still relay signed publishes from a trusted publisher unchanged, and any receiver along the way can verify the chain of custody.

Persistence is opt-in. The kernel's `PersistenceAdapter` contract exposes `write(namespace, value)` and `read(namespace)`; concrete adapters target IndexedDB (browser) and the filesystem (Node). When wired, AxonaPeer checkpoints identity, synaptome seeds, and pending subscriptions on a 5-second debounce. On restart, the identity

envelope is reloaded and validated (the stored `nodeId` must match the freshly-derived one); subscriptions are reinstalled as soon as the mesh has stabilised.

The `crypto.subtle` dependency is a hard requirement: the demo and the peer fail to boot in insecure contexts (HTTP pages outside localhost) because the Web Crypto API simply isn't available. The bridge serves over HTTPS; the peer's GitHub Pages deployment enforces HTTPS at the Pages level; the demo at `demo.axona.net` required Let's Encrypt cert provisioning before identity derivation would run at all.

Because identity is bound to a region prefix, moving geographic location changes the `nodeId`. `axona-peer`'s `identity.js` preserves the existing identity across reloads unless the user explicitly re-selects a different region. A roaming user keeps the same DHT identity for the life of the browser profile.

XIII Production Deployment

THE PUBLIC STACK RUNS AT THREE URLS.

bridge.axona.net. The deployed bridge. Runs on a DigitalOcean droplet, started by `systemd`, kept current via `git pull && npm ci --omit=dev && systemctl restart`. Exposes `/healthz` for monitoring. Listens on `wss://` only; HTTP requests are redirected.

axona.net. The reference peer application, served via GitHub Pages from the `axona-peer` repo's main branch. Every push redeploys. The vendored `axona-protocol` kernel ships with the app; the version row in the UI confirms which kernel an open tab is running.

demo.axona.net. The minimal pub/sub demo, served via GitHub Pages from the `axona-protocol` repo's main branch (where `examples/minimal-pubsub-browser/` lives). Same Pages deployment pattern; HTTPS-enforced because `crypto.subtle` requires a secure context.

A coordinated release works as follows: a fix lands in `axona-protocol`, bumps `KERNEL_VERSION` and the package version, tags the commit. `axona-peer` resyncs its `vendor/axona-protocol/` from the new kernel, bumps `PEER_VERSION`, and pushes (Pages picks up). The bridge updates its `@axona/protocol` dependency in `package.json`, bumps `VERSION` in `server.js`, pushes, and the operator runs the deploy script. Every component ends up reporting the same `kernelVersion` in its visible metadata.

XIV Future Directions

THE ARCHITECTURE IS STABLE. The shape of the layering, the wire format, the BigInt-canonical address discipline, the K-closest pub/sub semantics — these are not expected to change. What's coming sits inside the existing seams:

Every Axona component is required to surface its `KERNEL_VERSION` at a runtime-inspectable point: the bridge in `/healthz` and the welcome frame, the peer in its UI version row, the demo in its footer. The rule exists because before it did, “which kernel is everyone on?” required SSH-ing into the bridge and grepping `node_modules`.

Federated bridges. Multiple bridges in different geographic regions, gossiping peer-lists so a peer connected to one bridge can be reached from another. Single-bridge today is a deployment simplification, not a protocol constraint.

Demote the bridge to an ordinary peer. Federated bridges remove the single *rendezvous*; this companion change removes the single *super-peer*. The routing and pub/sub layers already treat a bridge as just another `nodeId` — there is no `isBridge` flag, no eviction exemption, no routing privilege.³ The residual special status is purely at the *bootstrap* layer: a peer holds its bridge `WebSocket` open for signaling and *auto-readmits* the bridge to its synaptome on every (re)connect, so the bridge never actually leaves the routing table. Once a peer holds enough healthy mesh peers it should *drop the bridge from routing and root candidacy* — keeping the socket open for signaling only — and stop auto-readmitting its `nodeId`. With several bridges sharing the load, this also dissolves the *correlated* failure mode in which one bridge restart removes a root from nearly every topic at once. The goal state: a bridge is a signaling endpoint that any operator can stand up, indistinguishable from any other peer to the routing fabric. Validate convergence and churn behaviour in `dht-sim` before shipping.

Bridge directory. A well-known public topic (`deriveTopicId(null, "axona.bridge-directory")`) where every bridge republishes itself once per hour: URL, region, kernel version, advertised TURN credentials, last-seen timestamp.⁴ Peers subscribe at startup and cache the directory; a peer whose primary bridge disappears has a list of alternates to try without out-of-band coordination. To prevent spam and DDoS of the channel, the topic uses an *allowed-signer set*: only Ed25519 keys with a valid attestation chain back to one of the genesis bridge keys (baked into the kernel) can publish. Subscribers verify the chain on every publish and drop unattested envelopes at the `AxonaManager` layer, before they reach the application. Genesis keys can sign attestations admitting new bridges without a kernel release. This turns “federated bridges” from a deployment story into something clients can actually discover at runtime.

Capability hardening (endo / SES). Adopt the Agoric and MetaMask object-capability stack to harden the host-side surface, in three landings: `@endo/marshal` for envelope payloads (so payloads can carry `BigInt`, remotable references, and the slot-and-body wire shape `captp` expects);⁵ an opt-in `AxonaPeer.create({secure: true})` that calls `lockdown()` to freeze JavaScript primordials before kernel init, defending against prototype pollution and supply-chain mutation; and `@endo/captp` layered over `peer.send/peer.onMessage`

³ The bridge’s present dominance is *emergent*, not granted: every peer dials it first, so it lands in every synaptome and carries the highest degree; and its `0x89` geo-identity sits XOR-close to every `us-east/*` topic. The ordinary K-closest and vitality rules then keep selecting it as a root. The topology makes it central; the algorithm gives it nothing.

⁴ Subscribers keep the newest entry per `signerPubkey` and prune entries older than 24 hours. Being a public topic, its ID carries the `0x00` prefix, so every peer derives the same directory ID and sees the same listing regardless of geography.

⁵ Marshal output is JSON-shaped with `@qclass` annotations on typed values, so legacy peers see plain objects and capability-aware peers see typed data. Negotiated per-envelope via a `payloadFormat` tag.

to upgrade direct messaging from string-handler RPC to capability-style RPC with promise pipelining and revocable handles. A later phase ships `@endo/import-bundle` for content-addressed, signature-verified code distribution — the substrate for running third-party agents and creator-published filters inside peer compartments with only the authority each receiver chose to grant.

Persistent topics. Today’s replay cache survives in-memory only.

A pluggable persistence layer for axon `role.replayCache` (with a TTL) lets late subscribers catch up across bridge restarts.

Topic access control. Public-mode topics (`publisher: null`) let anyone publish. Private topics (`publisher: <fixed-pubkey>`) accept publishes only from that key. A capability layer where subscribers verify per-publish signatures against an allowed-signer set lives at the application layer today; lifting it into the kernel is straightforward.

Cross-tab state sharing. Multiple tabs on the same browser today derive the same identity and form independent peers; a BroadcastChannel-based shared peer would reduce mesh load proportional to tab count.

Native transport. A C-based or Rust-based embedded kernel for IoT-class devices, sharing the wire format with the JS kernel so existing peers can talk to it without adaptation.

Connection-time optimization. The authenticated-identity handshake costs only sub-millisecond crypto (one Ed25519 sign plus one verify per link); the felt cost of a cold start is dominated instead by WebRTC ICE/DTLS bring-up, un-bundled module loading, and the few added round-trips of the mesh handshake — none of which touches steady-state routing or delivery latency. The recovery levers all sit at the client and transport layer, independent of the wire protocol: bundle the kernel modules, render the UI and authenticate in the background, and fold the mesh handshake into the DTLS/SDP setup so the signed hello rides the first data-channel frame.⁶ Not yet mission-critical; deferred until adoption makes first-connect time the binding constraint.

Black-hole detection (forwarding vitality). Every hardening to date answers “*is this peer who it claims to be, and may it do this?*” — identity, channel binding, ingress authorization, verified routing admission. A *black-hole node* passes all of them and then silently drops (or, equivalently, fails to forward) the traffic it should relay. This is a Byzantine behavioural fault, not an identity one. Tampering is already defeated (end-to-end signatures + A-1) and replay/re-order by C-2’s `seq`+freshness; *omission*, though, is only *inferable*,

⁶ The last lever pairs naturally with binding the channel-binding value to the DTLS certificate fingerprint: once the CBV derives from the fingerprint already exchanged in the SDP, no nonce round-trip is needed and the handshake adds no extra latency — security and speed land together.

never provable — you cannot prove a negative, and a malicious drop is indistinguishable from a crash or a bad link. The direction is a *local* forwarding-vitality signal — each node’s own first-party measurement of whether traffic routed through a peer reached an acked destination, over the existing K-closest path redundancy — driving route-around, never a gossiped reputation verdict (which the no-central-authority constraint forbids). Equivocation is the one bad-faith behaviour that *is* provable (signed conflicting artifacts) and can land earlier. The gating question — the false-positive rate against honest-but-flaky nodes — must be answered in simulation first; see the black-hole threat note and the heterogeneous-protocol simulation plan in **axona-docs**. This is the behavioural complement to verified admission: that governs *who is in your routing table*; this governs *whether they actually do their job*.

The job of the architecture is to make these additions land cleanly. The job of the existing layers is to keep working while they do.

Companion documents: the Axona Programmer Guide and Axona API Reference, the Axona Explainer, and the source repositories at github.com/axona-net.

A Appendix: The Demo Application

THIS IS THE COMPLETE SOURCE of the minimal-pubsub demo deployed at **demo.axona.net**. The file lives in the kernel repository under **examples/minimal-pubsub-browser/index.html**. Just over six hundred lines of HTML, CSS, and JavaScript, no build step, no dependencies outside the kernel barrel export. Open it in two tabs; one publishes, the other receives. The structure maps to the API reference in §XI section by section: identity, transport, peer construction, lifecycle, subscribe, publish — and, since kernel v2.1, the connection-state and ping-traffic hooks that drive the dot strip and survive a bridge restart.

```
examples/minimal-pubsub-browser/index.html
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, viewport-
    fit=cover">
  <meta name="theme-color" content="#2d7373">
  <title>Axona Demo</title>

  <style>
    :root {
      --bg:          #fffff8;
```

```

--ink:          #1a1a1a;
--muted:        #6b6b6b;
--card:         #ffffff;
--border:       #e2dfd6;
--accent:       #2d7373;
--accent-soft:  #d4e4e1;
--dot-bridge:   #2d7373;
--dot-open:     #22c55e;
--dot-pending:  #f59e0b;
--dot-dead:     #cbcbc4;
--tile-bg:      #f4f1e8;
}
* { box-sizing: border-box; }
html, body { margin: 0; padding: 0; background: var(--bg); color: var(--ink); }
body {
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
  line-height: 1.5;
  min-height: 100vh;
  padding: env(safe-area-inset-top) env(safe-area-inset-right) env(safe-area-inset-bottom) env(safe-area-inset-left);
}
.wrap {
  max-width: 880px;
  margin: 0 auto;
  padding: 1.4rem 1.1rem 3rem;
}

/* Header */
header {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 1rem;
  margin-bottom: 0.6rem;
}
h1 {
  font-size: 1.5rem;
  font-weight: 600;
  letter-spacing: -0.01em;
  margin: 0;
  color: var(--accent);
}
.status {
  font-size: 0.8rem;
  color: var(--muted);
  font-variant-numeric: tabular-nums;
}

/* Dot strip (mesh) */
.dots {
  display: flex;
  flex-wrap: wrap;
  gap: 4px;
  padding: 0.5rem 0 1rem;
  min-height: 18px;
}
.dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: var(--dot-dead);
  transition: background 0.15s, transform 0.15s, opacity 0.15s;
  opacity: 0.55; /* resting (no recent traffic) */
}
.dot.bridge { background: var(--dot-bridge); width: 9px; height: 9px; }
.dot.open { background: var(--dot-open); }
.dot.pending { background: var(--dot-pending); }

```

```

.dot.dead { background: var(--dot-dead); }

/* v2.0.2: pulse-on-ping. The dot brightens when a ping/pong
   frame crosses its channel. Class added on event, removed
   400 ms later → dot fades back to its resting opacity. Steady
   channels at 1 Hz visibly blink twice/second; stalled channels
   go dim within ~1 s. */
.dot.pulse {
  opacity: 1;
  transform: scale(1.35);
}

/* Topic info row */
.topic-row {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  margin-bottom: 1.2rem;
}
.tile {
  background: var(--tile-bg);
  border: 1px solid var(--border);
  border-radius: 6px;
  padding: 0.4rem 0.7rem;
  font-size: 0.82rem;
  color: var(--ink);
  display: flex;
  flex-direction: column;
  gap: 1px;
}
.tile .k { color: var(--muted); font-size: 0.7rem; letter-spacing: 0.04em;
  text-transform: uppercase; }
.tile .v {
  font-family: ui-monospace, 'SF Mono', Menlo, monospace;
  font-size: 0.86rem;
}
.tile.topic .v { color: var(--accent); font-weight: 600; }
.tile.prefix .v { color: var(--accent); font-weight: 600; }
.tile.topcid .v { color: var(--muted); font-size: 0.74rem; word-break:
  break-all; }

/* Two-pane grid */
.panes {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1rem;
}
@media (max-width: 700px) {
  .panes { grid-template-columns: 1fr; }
}
.pane {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 1rem 1.1rem;
}
.pane h2 {
  font-size: 0.78rem;
  font-weight: 600;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  color: var(--muted);
  margin: 0 0 0.7rem 0;
}

/* Publish form */
label.field {
  display: block;

```

```

    margin-bottom: 0.7rem;
}
label.field span {
    display: block;
    font-size: 0.74rem;
    color: var(--muted);
    margin-bottom: 0.2rem;
}
input[type="text"] {
    width: 100%;
    padding: 0.6rem 0.7rem;
    border: 1px solid var(--border);
    border-radius: 6px;
    font-family: inherit;
    font-size: 1rem;
    color: var(--ink);
    background: white;
    transition: border-color 0.15s, box-shadow 0.15s;
}
input[type="text"]:focus {
    outline: none;
    border-color: var(--accent);
    box-shadow: 0 0 0 3px var(--accent-soft);
}
button.publish {
    width: 100%;
    padding: 0.75rem 1rem;
    border: none;
    border-radius: 6px;
    background: var(--accent);
    color: white;
    font-family: inherit;
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.15s;
    margin-top: 0.3rem;
}
button.publish:hover { background: #1d5959; }
button.publish:active { background: #144545; }
button.publish:disabled { background: #9aafad; cursor: not-allowed; }

/* Subscribe feed */
.feed {
    min-height: 220px;
    max-height: 420px;
    overflow-y: auto;
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
}
.feed .empty {
    color: var(--muted);
    font-style: italic;
    font-size: 0.92rem;
    padding: 0.4rem 0;
}
.msg {
    padding: 0.55rem 0.7rem;
    background: var(--tile-bg);
    border-radius: 6px;
    font-size: 0.95rem;
}
.msg .meta {
    font-size: 0.72rem;
    color: var(--muted);
    margin-bottom: 0.15rem;
    display: flex;

```

```

        gap: 0.6rem;
        align-items: baseline;
    }
    .msg .name { font-weight: 600; color: var(--accent); }
    .msg .time { font-variant-numeric: tabular-nums; }
    .msg .body { color: var(--ink); white-space: pre-wrap; word-wrap: break-
word; }

/* Footer */
footer {
    margin-top: 1.6rem;
    padding-top: 0.9rem;
    border-top: 1px solid var(--border);
    font-size: 0.74rem;
    color: var(--muted);
    display: flex;
    justify-content: space-between;
    gap: 1rem;
    flex-wrap: wrap;
    font-family: ui-monospace, 'SF Mono', Menlo, monospace;
}
footer a { color: var(--accent); text-decoration: none; }
footer a:hover { text-decoration: underline; }
</style>
</head>
<body>
<div class="wrap">

    <header>
        <h1>Axona Demo</h1>
        <div class="status" id="status">...connecting</div>
    </header>

    <div class="dots" id="dots" aria-label="connected peers"></div>

    <div class="topic-row">
        <div class="tile topic">
            <span class="k">Topic</span>
            <span class="v">hello-world</span>
        </div>
        <div class="tile prefix">
            <span class="k">Anchor</span>
            <span class="v" id="prefix">-</span>
        </div>
        <div class="tile topicid">
            <span class="k">Topic ID</span>
            <span class="v" id="topicId">-</span>
        </div>
    </div>

    <div class="panes">
        <div class="pane">
            <h2>Publish</h2>
            <label class="field">
                <span>Name</span>
                <input type="text" id="nameInput" value="Alice" autocomplete="off"
autocapitalize="words">
            </label>
            <label class="field">
                <span>Message</span>
                <input type="text" id="msgInput" value="Hello, world." autocomplete="
off">
            </label>
            <button class="publish" id="pubBtn" disabled>Publish</button>
        </div>

        <div class="pane">
            <h2>Subscribe</h2>

```

```

    <div class="feed" id="feed">
      <span class="empty">Waiting for ...messages</span>
    </div>
  </div>

  <footer>
    <span>demo v<span id="demoVer">...</span> · kernel v<span id="kernelVer">...</span></span>
    <a href="https://github.com/axona-net/axona-protocol">source</a>
  </footer>

</div>

<script type="module">
import {
  AxonaPeer, AxonaDomain, NeuronNode,
  deriveIdentity, deriveTopicId, geoCellId,
} from '/src/index.js';
import { webTransport } from '/src/transport/web/index.js';

// Constants
// Bump the middle digit on every code change to this file so the
// footer reflects what's actually running and stale browser tabs are
// distinguishable.
const DEMO_VERSION = '1.6.0';
const BRIDGE_URL = new URLSearchParams(window.location.search).get('bridge')
  || 'wss://bridge.axona.net';
const VIRGINIA = { lat: 38.0, lng: -77.0 };
// axona-peer composes topic names as `${regionId}/${eventName}` (see
// axona-peer/src/client.js line 1230). Use the same composition here
// so the demo's hello-world topic at us-east is the EXACT topic that
// axona-peer subscribers on us-east/hello-world see. Without this,
// the 0x89 anchor matches but the body hash differs and the two apps
// never converge. EVENT is what we show in the UI tile; TOPIC is
// what crosses the wire.
const REGION_ID = 'us-east';
const EVENT = 'hello-world';
const TOPIC = `${REGION_ID}/${EVENT}`;
const MAX_DOTS = 100;

// DOM
const $dots = document.getElementById('dots');
const $status = document.getElementById('status');
const $prefix = document.getElementById('prefix');
const $topicId = document.getElementById('topicId');
const $kernel = document.getElementById('kernelVer');
const $name = document.getElementById('nameInput');
const $msg = document.getElementById('msgInput');
const $pubBtn = document.getElementById('pubBtn');
const $feed = document.getElementById('feed');

// Footer versions
document.getElementById('demoVer').textContent = DEMO_VERSION;
try {
  const pkg = await fetch('/package.json').then(r => r.json());
  $kernel.textContent = pkg.version;
} catch { $kernel.textContent = '?'; }

// Topic info (Virginia-anchored)
const virginiaPrefix = geoCellId(VIRGINIA.lat, VIRGINIA.lng, 8);
$prefix.textContent = '0x' + virginiaPrefix.toString(16).padStart(2, '0').
  toUpperCase();

// Synthetic publisher: nodeId-shaped hex whose top 8 bits = Virginia
// S2 cell. Anchors the topic at Virginia regardless of where the
// subscribing peer lives - every demo user worldwide computes the
// same topicId.

```

```

const publisher = virginiaPrefix.toString(16).padStart(2, '0') + '0'.repeat
(64);
const topicId = await deriveTopicId(publisher, TOPIC);
$topicId.textContent = topicId.slice(0, 12) + ...'' + topicId.slice(-6);

// Geolocation (for our own identity only)
async function getLocation() {
  // Even if denied, identity falls back to (0, 0) → cell 0x10 (Gulf
  // of Guinea). We never end up with a 0x00 identity prefix unless
  // the user is at the South Pole. The topic anchor above is
  // independent of this - it always stays at Virginia (0x89).
  const FALLBACK = { lat: 0, lng: 0 };
  if (!navigator.geolocation) return FALLBACK;
  return new Promise(resolve => {
    navigator.geolocation.getCurrentPosition(
      pos => resolve({ lat: pos.coords.latitude, lng: pos.coords.longitude })
    ,
      () => resolve(FALLBACK),
      { enableHighAccuracy: false, timeout: 6000, maximumAge: 600_000 },
    );
  });
}
const geo = await getLocation();

// Identity + transport + peer
const identity = await deriveIdentity({ lat: geo.lat, lng: geo.lng });
const transport = webTransport({ bridgeUrl: BRIDGE_URL, identity });
const node = new NeuronNode({ id: BigInt('0x' + identity.id), lat: geo.lat
, lng: geo.lng });
node.transport = transport;
const domain = new AxonaDomain({ k: 20 });
const peer = new AxonaPeer({ domain, node, identity, transport });

// Peer-state tracking for the dot strip
//
// We render one dot per known peer. Bridge is always the first
// (larger, accent-colored) dot, drawn whenever the bridge connection
// is alive. Each subsequent dot is a WebRTC peer.
//
// open    - green    (bound peer, isConnected = true)
// pending - amber    (in synaptome but not yet bound - fresh signal)
// dead    - grey     (was bound, transport reported death)
//
// Cap at MAX_DOTS; if more peers, we just truncate (rare in a single
// browser tab - typically ~520 connections).
// Render the dot strip directly from transport.boundPeers() - the
// transport's own ground truth. This matches axona-peer's pattern
// (client.js:455 - for each render tick it diffs mesh.peers()
// against the existing rows and DOM-removes anything no longer
// present). There's no separate death-tracking map: a peer that
// dies just stops appearing. No grey-dot accumulation.
let bridgeUp = false;

// Transport callbacks deliver peerIds as BigInts (kernel canonical
// form). Normalise to lowercase 66-char hex for the dot's tooltip.
function asHex(peerId) {
  if (typeof peerId === 'bigint') return peerId.toString(16).padStart(66, '0')
  ;
  if (typeof peerId === 'string') return peerId.toLowerCase();
  return String(peerId);
}

// Map peerHex → its current dot DOM node, so onPingTraffic can flash
// the right dot without re-walking the children every event. Cleared
// on each renderDots(); rebuilt as we create children.
const dotByPeer = new Map();
let bridgeDot = null;

```

```

function renderDots() {
  $dots.innerHTML = '';
  dotByPeer.clear();
  // Bridge dot first
  bridgeDot = document.createElement('div');
  bridgeDot.className = 'dot bridge' + (bridgeUp ? ' open' : ' pending');
  bridgeDot.title = 'bridge';
  $dots.appendChild(bridgeDot);
  // Currently-bound peers (kernel reports only live channels)
  const peers = [...(transport.boundPeers?.() || [])].slice(0, MAX_DOTS - 1);
  for (const peerId of peers) {
    const hex = asHex(peerId);
    const d = document.createElement('div');
    d.className = 'dot open';
    d.title = hex.slice(0, 12) + ...;
    $dots.appendChild(d);
    dotByPeer.set(hex, d);
  }
}
renderDots();

// v2.0.2: pulse a dot on every ping-sent / pong-received frame on
// that peer's channel. Class is removed 400 ms later so a steady
// 1 Hz traffic looks like a soft, regular blink and a stalled
// channel goes dim within a second.
function pulse(dot) {
  if (!dot) return;
  dot.classList.add('pulse');
  // clearTimeout existing handle if any, to avoid premature class removal
  // when two events arrive in quick succession.
  if (dot._pulseTimer) clearTimeout(dot._pulseTimer);
  dot._pulseTimer = setTimeout(() => dot.classList.remove('pulse'), 400);
}

transport.onPingTraffic?.((peerId /*, kind */) => {
  const hex = asHex(peerId);
  // Bridge ping/pong attributes to the bridge's nodeId - the same
  // hex appears in boundPeers() as the bridge connection. If the
  // hex matches the bridge's nodeId, pulse the bridge dot; otherwise
  // look up the per-peer dot.
  const bridgeHex = transport.bridgeNodeId ? asHex(transport.bridgeNodeId) :
    null;
  if (bridgeHex && hex === bridgeHex) pulse(bridgeDot);
  else pulse(dotByPeer.get(hex));
});

// onPeerBound / onPeerDied are just render triggers - boundPeers()
// is the source of truth, so we re-read it on every event.
transport.onPeerBound?.(() => renderDots());
transport.onPeerDied?.(() => renderDots());

// Connect
//
// Two-phase: transport+peer start first (handshake with the bridge),
// then wait for the mesh to fill in before subscribing. Subscribing
// while synaptome.size === 1 (just the bridge) registers our
// subscription at the bridge only. When other peers come online and
// other publishers route their `pubsub:publish-k` through a fuller
// K-closest set, those publishes can miss us because our subscription
// is anchored at the wrong axon. Gating the subscribe behind a
// stable synaptome matches axona-peer's manual "Add subscription"
// pattern and what dht-sim does after warm-up.
//
// READY_SYNAPSE_COUNT = bridge + ~3 WebRTC peers; READY_TIMEOUT_MS
// puts a ceiling on solo-tab runs (when there's no one else to find).
const READY_SYNAPSE_COUNT = 4;
const READY_TIMEOUT_MS = 10_000;

```



```

async function waitForMeshReady() {
  const t0 = Date.now();
  while (Date.now() - t0 < READY_TIMEOUT_MS) {
    if (node.synaptome.size >= READY_SYNAPSE_COUNT) return node.synaptome.size;
    ;
    await new Promise(r => setTimeout(r, 200));
  }
  return node.synaptome.size;
}

try {
  await transport.start(identity.id);
  await peer.start();
  bridgeUp = true;
  $status.textContent = 'stabilising ...mesh';
  renderDots(); // bridge dot + any peers already bound

  await waitForMeshReady();
  // Subscription happens just below (top-level await) - gate the
  // button on success after sub resolves.
} catch (err) {
  $status.textContent = 'connection failed';
  console.error('[demo] connect failed', err);
}

// Subscribe
let messageCount = 0;
function appendMessage(name, body, when) {
  if (messageCount === 0) $feed.innerHTML = '';
  messageCount++;
  const msg = document.createElement('div');
  msg.className = 'msg';
  const meta = document.createElement('div');
  meta.className = 'meta';
  const nameEl = document.createElement('span');
  nameEl.className = 'name';
  nameEl.textContent = name;
  const timeEl = document.createElement('span');
  timeEl.className = 'time';
  timeEl.textContent = when.toLocaleTimeString([], { hour: '2-digit', minute:
    '2-digit' });
  meta.appendChild(nameEl);
  meta.appendChild(timeEl);
  const bodyEl = document.createElement('div');
  bodyEl.className = 'body';
  bodyEl.textContent = body;
  msg.appendChild(meta);
  msg.appendChild(bodyEl);
  $feed.appendChild(msg);
  // Auto-scroll to the newest message
  $feed.scrollTop = $feed.scrollHeight;
}

const sub = await peer.sub(TOPIC, (envelope) => {
  // Envelopes from the wire are "Name: Body" strings.
  const wire = envelope.message || '';
  const colonAt = wire.indexOf(':');
  const name = colonAt > 0 ? wire.slice(0, colonAt).trim() : '(unknown)';
  const body = colonAt > 0 ? wire.slice(colonAt + 1).trim() : wire;
  appendMessage(name, body, new Date());
}, { publisher, since: 'all' });

// Arm the kernel's AxonaManager periodic refresh timer. By default
// AxonaPeer._buildDefaultAxonaManager() does NOT call am.start() (the
// comment in AxonaPeer.js:1820 says "applications call peer.sub after
// the mesh has stabilised, so the initial K-closest is already wide
// and refresh isn't needed"). That assumption holds for the dht-sim
// simulator's steady-state runs but NOT for a browser mesh, where

```

```

// WebRTC peers come and go on every page reload. Without this, a
// subscription registered at the initial K-closest set becomes stale
// as the mesh churns, and publishes routed through a new K-closest
// set miss the stale subscribers. Arming refreshTick at the 10 s
// default keeps the subscription pinned to whoever is K-closest
// right now.
peer._axonaManager?.start?();

// Backstop re-render every 2 s. onPeerBound / onPeerDied catch
// most transitions, but some peers die silently (network drop, page
// suspended, browser throttled tab). A periodic re-read of
// transport.boundPeers() keeps the strip honest without a
// state-tracking map.
setInterval(renderDots, 2000);

// Resume recovery (laptop sleep / tab background)
// The kernel transport already self-heals after a connection freeze:
// dead WebRTC channels are evicted on heartbeat timeout (mesh v2.1.1)
// and the bridge socket auto-reconnects with backoff. This is the
// optional *application-layer* accelerator the peer at axona.net also
// uses - on a genuine resume event, tear the mesh down at once
// (mesh.reset() fires onPeerLost so the synaptome clears too) and force
// an immediate bridge reconnect, so recovery is snappy rather than
// waiting out the per-peer timeouts. A macOS screensaver may fire NONE
// of these events (the tab never goes hidden); that's exactly the case
// the kernel-side eviction backstops.
function onResume(reason) {
  console.log('[demo] resume:', reason);
  try { transport.mesh?.reset?(); } catch { /* mesh may already be gone */ }
  transport.reconnectNow?();
}
let hiddenAt = 0;
document.addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'hidden') { hiddenAt = Date.now(); return;
  }
  if (hiddenAt && Date.now() - hiddenAt >= 5000) onResume('visible');
  hiddenAt = 0;
});
window.addEventListener('online', () => onResume('online'));
window.addEventListener('pageshow', (e) => { if (e.persisted) onResume('
  bfcache'); });

// Subscription is registered - safe to publish now.
$status.textContent = 'connected';
$pubBtn.disabled = false;

// Publish
async function publish() {
  const name = ($name.value || 'Alice').trim();
  const msg = ($msg.value || 'Hello, world.').trim();
  if (!msg) return;
  $pubBtn.disabled = true;
  try {
    await peer.pub(TOPIC, name + ': ' + msg, { publisher });
  } catch (err) {
    console.error('[demo] publish failed', err);
  } finally {
    $pubBtn.disabled = false;
    $msg.focus();
    $msg.select();
  }
}
$pubBtn.addEventListener('click', publish);
$msg.addEventListener('keydown', (e) => {
  if (e.key === 'Enter' && !$pubBtn.disabled) publish();
});
</script>

```

```
</body>  
</html>
```